
ADAPTIVE GRAPH-OF-ISLANDS EVOLUTION FOR AUTOMATIC FEATURE ENGINEERING WITH LLMs

Sha Li

Department of Computer Science
Virginia Tech
Blacksburg, VA 24060
shal@vt.edu

Naren Ramakrishnan

Department of Computer Science
Virginia Tech
Alexandria, VA 22305
naren@cs.vt.edu

September 3, 2026

ABSTRACT

Automatic feature engineering (AutoFE) for tabular data requires discovering informative transformations from a combinatorially large and heterogeneous program space. Existing approaches suffer from three fundamental limitations: classical methods rely on fixed operator libraries with limited expressivity, LLM-based methods generate proposals from static prompts without retaining prior search experience, and evolutionary methods employ fixed migration policies that ignore task-specific cross-family transfer utility. We introduce TOPOFE, a framework that formulates AutoFE as graph-structured multi-island evolutionary program search. The transformation space is partitioned into semantically coherent families, each explored by a dedicated island through LLM-guided mutation and crossover. Each island maintains a Prompt Adaptation Memory that accumulates accept/reject feedback to steer future proposals toward productive regions of the search space without parameter updates. To coordinate global exploration, TOPOFE dynamically learns a directed topology graph whose edge weights encode empirical transfer utility between transformation families. Cross-island transfer is triggered by adaptive saturation detection and performed through LLM-mediated hybrid synthesis, enabling discovery of compositional feature programs that cannot emerge from isolated local search. Experiments on 29 tabular datasets show that TOPOFE consistently outperforms most state-of-the-art AutoFE methods on both classification and regression tasks. Beyond predictive performance, TOPOFE produces feature sets with lower redundancy and higher representational coverage, while the learned topology graph acquires meaningful task-specific transfer structure correlated with downstream gains. The discovered feature programs also transfer reliably across diverse downstream predictors and LLM backbones, demonstrating that the improvements arise from TOPOFE’s structured search and adaptive coordination mechanisms instead of backbone-specific generation capability.

1 Introduction

Feature engineering (FE) is the process of transforming raw variables into predictive representations that expose the latent structure of a learning problem [Dong and Liu, 2018], it remains one of the most consequential yet least principled stages of the tabular machine learning pipeline. In tabular settings, where data are heterogeneous, semantically rich, and often laden with domain-specific structure, the quality of engineered features frequently determines model performance more than architectural choices or hyperparameter tuning [Zhang et al., 2023a, Hollmann et al., 2023a]. Constructing high-value features requires more than syntactic enumeration. For example, a clinically meaningful risk ratio, a financial volatility indicator, or a behavioral frequency feature demands understanding of what variables mean, how they causally interact, and what transformations are semantically admissible. This semantic knowledge lies beyond what fixed operator libraries can encode.

Early automated feature engineering (AutoFE) systems frame the problem as search over a predefined transformation grammar [Olson and Moore, 2016, Horn et al., 2019, Zhang et al., 2023a]. While tractable, this formulation is

fundamentally bounded by the expressiveness of the operator library: features requiring cross-variable semantic reasoning lie outside the reachable space by construction. The emergence of large language models (LLMs) introduced a qualitatively different capability: by pretraining on massive corpora of scientific, mathematical, and domain-specific text, LLMs can hypothesize meaningful transformations from column names, task descriptions, and background knowledge, effectively shifting the paradigm from fixed operator selection to open-ended feature program synthesis [Hollmann et al., 2023a, Han et al., 2024]. However, existing LLM-based methods share a critical structural flaw: proposals are generated from static prompts with no memory of prior evaluations, leading to repeated proposals, no adaptation to dataset-specific patterns, and no mechanism to synthesize features requiring operators from multiple transformation families simultaneously.

Recent effort of combining LLMs with evolutionary search partially mitigates this limitation by maintaining an experience buffer of high-scoring programs as in-context demonstrations [Abhyankar et al., 2025, Gong et al., 2025]. Yet even these methods operate over a single undifferentiated population. The feature program space is not homogeneous, it comprises qualitatively distinct transformation families (arithmetic interactions, statistical aggregates, temporal dynamics, relational encodings, nonlinear univariate maps), each encoding a different inductive bias about how predictive signal arises. Single-population search ignores this structure. Once a few successful programs from a dominant family saturate the experience buffer, subsequent LLM proposals become progressively confined to that family. Orthogonal families are under-explored, and cross-family compositions, which often yield the most expressive and complementary features, are structurally unreachable. This *cross-family lock-in* is the central failure mode of existing LLM-evolutionary methods, and no existing work provides a principled mechanism to detect or correct it. Three interrelated limitations persist across the literature. *(L1) Homogeneous search dynamics:* single-population evolution collapses onto a narrow subset of transformation types, biased toward early high-performing motifs and blind to the multi-modal structure of program space. *(L2) Stateless LLM querying:* feature generation lacks persistent memory of prior exploration, preventing cumulative adaptation to dataset-specific patterns. *(L3) Rigid transfer structure:* when multiple populations are used, migration is periodic or random, agnostic to when transfer is beneficial and which source family is most complementary to a stagnating target island.

We introduce TOPOFE, a framework that models feature engineering as program search inspired by graph-structured multi-island evolution. TOPOFE rests on three principles. First, *inductive bias decomposition*: the program space is partitioned into different transformation families, each assigned a dedicated island that evolves specialized programs under family-specific priors, preserving diversity across transformation types by construction. Second, *adaptive topology learning*: instead of fixed or heuristic migration schedules, TOPOFE maintains a directed weighted graph over islands whose edge weights are updated online from observed cross-family transfer gains, learning a data-driven model of inter-family complementarity. Third, *semantic transfer via LLM synthesis*: cross-island knowledge transfer is not realized by copying programs, but through LLM-mediated hybrid synthesis in which programs from a source and target island are jointly presented as context, and the LLM generates a compositionally novel program that inherits structural elements from both families. Transfer is triggered by a *saturation criterion*: a sliding-window estimate of each island’s marginal improvement that detects when local search has genuinely exhausted its productive region. This decouples transfer from wall-clock generation count and ensures that the evaluation budget is directed toward cross-family exploration precisely when it is most valuable. A *Prompt Adaptation Memory* component further maintains a per-island accept/reject history and a compact natural-language summary of preferred and avoided transformation patterns, injected into every LLM call. This enables cumulative, dataset-specific adaptation throughout the search without any parameter update.

We summarize our contributions as follows: **(i) Topology-aware formulation.** We formulate AutoFE as graph-structured multi-island program search, partitioning the feature space into family-specialized islands and replacing heuristic migration with a learned directed topology graph whose edge weights encode empirically observed cross-family transfer utility, updated online via a bandit-style rule. **(ii) Saturation-triggered adaptive transfer.** We introduce a utility-based saturation criterion that triggers cross-island transfer when local search is exhausted. A per-island Prompt Adaptation Memory accumulates accepted and rejected program patterns, enabling cumulative dataset-specific adaptation throughout search without any parameter update. **(iii) Principled evaluation framework.** We introduce three purpose-built metrics: Mean Pairwise Output Correlation and Effective Rank jointly characterise feature redundancy and subspace coverage, and Topology Graph Specialisation Score quantifies the degree to which the topology graph acquires task-specific cross-family transfer knowledge during search. **(iv) Empirical validation.** Across 29 datasets we demonstrate that TOPOFE consistently outperforms most classical and LLM-based AutoFE baselines, producing lower-redundancy and higher-coverage feature sets, with performance remaining stable across downstream predictors and LLM backbones of varying capability.

2 Preliminary

2.1 Feature Engineering (FE)

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ be a tabular dataset with N instances, d -dimensional feature vector $\mathbf{x}_i \in \mathcal{X} \subseteq \mathbb{R}^d$ and target variable $y_i \in \mathcal{Y}$. Each dataset is accompanied by structured metadata $\mathcal{M} = \{\mathcal{M}^{\text{task}}, \mathcal{M}^{\text{feat}}\}$, encoding task type and per-feature semantic descriptors. Given a feature transformation $\mathcal{T} : \mathcal{X} \rightarrow \tilde{\mathcal{X}}$, let $\mathcal{F}$ denote the class of downstream predictive models and $\mathcal{L}$ a task-appropriate loss function, the optimal downstream model is obtained via empirical risk minimization on training split $\{\mathbf{X}_{\text{tr}}, \mathbf{y}_{\text{tr}}\}$:

$$f_{\mathcal{T}}^* = \arg \min_{f \in \mathcal{F}} \mathcal{L}(f(\mathcal{T}(\mathbf{X}_{\text{tr}})), \mathbf{y}_{\text{tr}}). \quad (1)$$

The *feature engineering objective* is to find the transformation $\mathcal{T}^*$ that maximizes generalization performance on held-out data:

$$\mathcal{T}^* = \arg \max_{\mathcal{T}} \mathbb{E}[\text{Perf}(f_{\mathcal{T}}^*, \mathcal{T}(\mathbf{X}_{\text{val}}), \mathbf{y}_{\text{val}})], \quad (2)$$

where Perf is a task-appropriate metric (e.g., accuracy). This constitutes a *bilevel optimization*: the inner problem fits the model under a fixed transformation, while the outer searches for the transformation itself. As the outer objective is non-differentiable with respect to $\mathcal{T}$ and each evaluation requires re-fitting f from scratch, sample-efficient search strategies are essential.

2.2 AutoFE as LLM-guided Program Search

We instantiate the transformation space $\mathcal{T}$ as a space of executable feature programs over a finite typed *operator library* $\mathcal{O} = \{o_q\}_{q=1}^Q$, where each operator o_q has a specified arity and type signature (e.g., $+$, $-$, $\times$, $\div$, $\log$, $\exp$). A *feature program* $p \in \mathcal{P}$ is any function $p : \mathcal{X} \rightarrow \mathbb{R}$ obtained by composing operators from $\mathcal{O}$ up to a bounded depth L_{max} :

$$\mathcal{P} := \left\{ p \mid p = o^{(L)} \circ \dots \circ o^{(1)}, o^{(\ell)} \in \mathcal{O}, L \leq L_{\text{max}} \right\}. \quad (3)$$

Each $p \in \mathcal{P}$ is represented as an executable Python function. A feature set $S \subseteq \mathcal{P}$ induces a transformed design matrix $\mathcal{T}_S(\mathbf{X}) = [p(\mathbf{X})]_{p \in S} \in \mathbb{R}^{N \times |S|}$, where each column corresponds to the output of one feature program. The optimal downstream model is:

$$f_S^* = \arg \min_{f \in \mathcal{F}} \mathcal{L}(f(\mathcal{T}_S(\mathbf{X}_{\text{tr}})), \mathbf{y}_{\text{tr}}). \quad (4)$$

The feature engineering objective can therefore be formulated as a *combinatorial program search* problem:

$$S^* = \arg \max_{S \subseteq \mathcal{P}} \mathbb{E}[\text{Perf}(f_S^*, \mathcal{T}_S(\mathbf{X}_{\text{val}}), \mathbf{y}_{\text{val}})]. \quad (5)$$

This search is intractable by enumeration because $|\mathcal{P}|$ grows super-exponentially with L_{max} , and non-differentiable with respect to discrete program structure. Each evaluation incurs cost $\mathcal{O}(k_{\text{cv}} \cdot C_{\text{fit}}(N, |S|))$ under k_{cv} -fold cross-validation, where $C_{\text{fit}}(N, |S|)$ denotes the training cost on N instances and $|S|$ engineered features. The fitness signal $\hat{\Phi}(S)$ is estimated as:

$$\hat{\Phi}(S) = \frac{1}{k_{\text{cv}}} \sum_{k=1}^{k_{\text{cv}}} \text{Perf}(f_{S,k}^*, \mathcal{T}_S(\mathbf{X}_k^{\text{val}}), \mathbf{y}_k^{\text{val}}), \quad (6)$$

where $f_{S,k}^*$ is trained on the k -th training fold and evaluated on the corresponding held-out fold $\mathbf{X}_{\text{val}}, \mathbf{y}_{\text{val}}$. The metadata $\mathcal{M}$ restricts $\mathcal{P}$ to semantically valid programs and provides a natural-language prior exploited directly by LLM-guided synthesis.

3 TOPOFE

Based on the preliminary stated in § 2.2, we argue that, combinatorial program search over $\mathcal{P}$ is intractable by enumeration and inaccessible to gradient-based methods, requiring a LLM-guided proposal mechanism that satisfies three properties: (i) generate syntactically valid and semantically meaningful programs over a heterogeneous operator space without exhaustive enumeration, (ii) exploit task-specific domain knowledge in $\mathcal{M}$ to bias search toward promising and admissible transformations, and (iii) support diverse proposal modes within a unified interface, enabling both exploitation of known high-quality programs and exploration of unvisited regions of $\mathcal{P}$. To satisfy these requirements, we propose TOPOFE, a framework that organizes LLM-guided program search for AutoFE through three coordinated

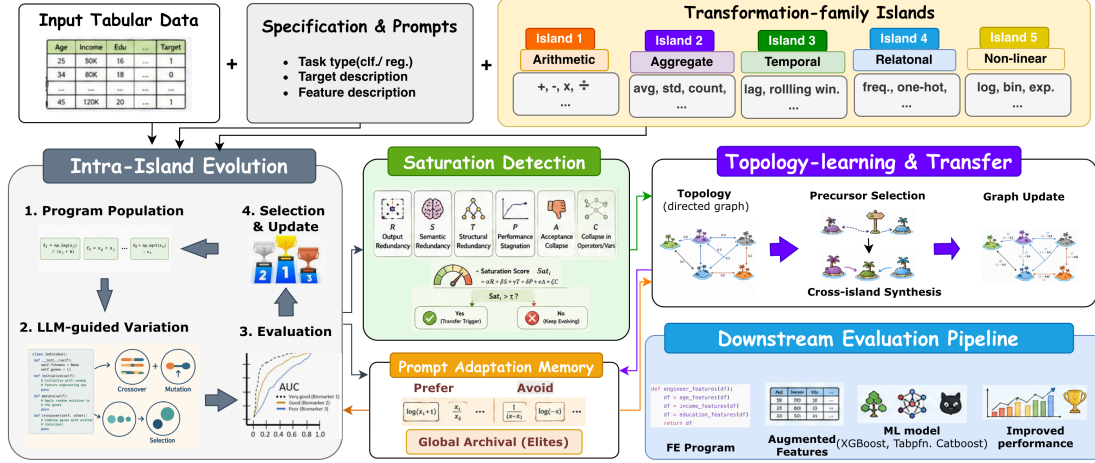


Figure 1: Overview of TOPOFE. It decomposes FE-program search into multiple transformation-family islands with LLM-guided local evolution, adaptive saturation detection, topology-aware cross-island synthesis, and archive-based selection to generate diverse engineered features that improve downstream tabular prediction performance.

components. First, a *multi-island decomposition* partitions $\mathcal{P}$ into M semantically coherent transformation families, each explored by a dedicated island to prevent premature convergence to a single family (§3.1). Second, *Prompt Adaptation Memory* maintains per-island search state that continuously adapts LLM proposals from accumulated accept/reject history without any parameter update (§3.2). Third, a *topology-aware cross-island transfer* mechanism governs knowledge flow across islands via a learned directed graph, triggering hybrid synthesis from complementary families when local search saturates (§3.3). Together, these components balance specialized exploitation within transformation families and structured exploration across them. The overview of TOPOFE is shown in Figure 1 and the complete procedure is summarized in Algorithm 1.

3.1 Intra-island Evolution

3.1.1 Multi-island Decomposition

The feature program space $\mathcal{P}$ is not homogeneous: programs naturally cluster into semantically coherent regions according to the class of operations they employ, each encoding a distinct inductive bias about how predictive signal is constructed from raw features.

Definition 1 (Transformation Family). A transformation family $\mathcal{P}_i \subseteq \mathcal{P}$ is a semantically coherent subset of feature programs sharing the same inductive bias about how predictive signal is constructed from raw features, i.e., programs that apply the same class of primitive operations.

We consider five canonical transformation families: arithmetic interactions $\mathcal{P}_1$ (products, ratios, and differences), statistical aggregates $\mathcal{P}_2$ (group-level means, standard deviations, counts, and medians), temporal features $\mathcal{P}_3$ (lags, rolling windows, and exponential moving averages), relational encodings $\mathcal{P}_4$ (frequency counts, target encodings, and rank-based transforms), and nonlinear univariate maps $\mathcal{P}_5$ (logarithmic, exponential, binned, and polynomial transformations). These families may overlap (i.e., $\mathcal{P}_i \cap \mathcal{P}_j \neq \emptyset$ in general) but each represents a structurally distinct region of $\mathcal{P}$. The partition is also extensible to additional transformation families as needed. Together, they induce an approximate cover $\mathcal{P} \approx \bigcup_{i=1}^M \mathcal{P}_i$, which decomposes the monolithic search space into M semantically typed subspaces, each amenable to specialized search.

However, in a single evolutionary population, high-scoring programs from a dominant family rapidly saturate the experience buffer, confining subsequent LLM proposals to that family’s neighbourhood and leaving orthogonal families permanently unexplored. TOPOFE addresses this through a structured multi-island decomposition that assigns a dedicated subpopulation to each $\mathcal{P}_i$, preserving family-specific inductive biases and preventing any single family from monopolizing the search budget.

Definition 2 (Island). Island i at generation t is a tuple $\mathcal{I}_i^{(t)} = (\Pi_i^{(t)}, \mathcal{A}_i, \mathcal{H}_i^{(t)}, \rho_i^{(t)})$, where: (i) $\Pi_i^{(t)} \subset \mathcal{P}_i$ is the current population of at most $n_{\max}$ feature programs, ranked by fitness $\hat{\Phi}$, (ii) $\mathcal{A}_i \subset \mathcal{P}_i$ is a long-term archive of elite

programs discovered by island i , (iii) $\mathcal{H}_i^{(t)}$ is the accept/reject history used to adapt the mutation prompt, and (iv) $\rho_i^{(t)} \in \mathbb{R}^s$ is a prompt-memory vector encoding a compact “prefer/avoid” signal derived from $\mathcal{H}_i^{(t)}$.

These four components maintain the island’s complete search state: $\Pi_i^{(t)}$ drives local search, $\mathcal{A}_i^{(t)}$ accumulates the best discoveries and serves as the in-context demonstration pool for LLM proposals, $\mathcal{H}_i^{(t)}$ provides the raw signal for prompt adaptation, and $\rho_i^{(t)}$ translates that signal into natural-language guidance injected into every LLM call.

3.1.2 LLM-guided Intra-island Evolution (Specialized Exploration)

At each generation t , island i independently evolves its population via two LLM-guided operators, both conditioned on the island’s metadata context $\mathcal{M}$ and prompt memory $\rho_i^{(t)}$:

- **Mutation** samples a single parent $p \sim \Pi_i^{(t)}$ with probability proportional to $\hat{\Phi}(p)$ and prompts the LLM to produce a modified program $p' \in \mathcal{P}_i$ that improves on p while respecting the signals encoded in $\rho_i^{(t)}$.
- **Crossover** samples two parent programs $(p_a, p_b) \sim \Pi_i^{(t)}$ and presents them as in-context demonstrations. The LLM is then prompted to synthesize a child program $p' \in \mathcal{P}_i$ that combines compositional elements from both parents.

Both operators pass through a unified validation and fitness pipeline: each candidate p' is checked for syntactic validity and semantic admissibility under $\mathcal{M}$, then evaluated via oracle $\hat{\Phi}$. The population $\Pi_i^{(t+1)}$ retains the top- $n_{\max}$ programs by fitness, and $\mathcal{A}_i^{(t)}$ and $\rho_i^{(t)}$ are updated following the rules defined in §3.2.

3.1.3 Global Objective

The multi-island system assembles the final feature set by pooling elites across all islands:

$$S^* = \arg \max_{S \subseteq \bigcup_i \mathcal{A}_i} \hat{\Phi}(S), \quad \text{s.t. } |S| \leq d'_{\max}, \quad \sum_{i=1}^K |\Pi_i^{(t)}| \leq B, \quad (7)$$

where B is the total evaluation budget and each retained program must contribute non-redundant predictive signal: $\hat{\Phi}(\{p\} \mid S^* \setminus \{p\}) > 0 \forall p \in S^*$.

3.2 Prompt Adaptation Memory

While the LLM’s pretrained knowledge provides a strong prior over feature programs, effective search requires the proposal mechanism to *adapt online* to each island’s evolving landscape. Without adaptation, the LLM repeatedly proposes programs in regions already known to be unproductive, wasting evaluation budget. We introduce *Prompt Adaptation Memory* (PAM), a lightweight per-island mechanism that continuously shapes the LLM’s proposal distribution from accumulated search experience without any parameter update. PAM operates through three coordinated update rules applied after every generation.

3.2.1 Prompt-memory Update

Let $\mathcal{H}_i^+(t)$ and $\mathcal{H}_i^-(t)$ denote the accepted and rejected subsets of the sliding history window $[t - W, t]$. Operator types concentrated among accepted programs are reinforced as *prefer* signals, those concentrated among rejected programs are flagged as *avoid* signals. Formally:

$$\rho_i^{(t+1)} = \text{Summarize}(\mathcal{H}_i^+(t), \mathcal{H}_i^-(t), \rho_i^{(t)}), \quad (8)$$

where $\text{Summarize}(\cdot)$ is an LLM call that produces a compact natural-language string (e.g., “prefer ratio features with lagged denominators, avoid log transforms of sparse columns”) prepended to all subsequent prompts for island i .

3.2.2 Elite Archive Update

After each generation, $\mathcal{A}_i^{(t)}$ merges newly accepted programs, re-ranks by $\hat{\Phi}$, and truncating to capacity $|\mathcal{A}|_{\max}$:

$$\mathcal{A}_i^{(t+1)} = \text{TopK}(\mathcal{A}_i^{(t)} \cup \{p : a_t = 1\}, |\mathcal{A}|_{\max}, \hat{\Phi}). \quad (9)$$

Candidates whose normalized tree-edit distance to any existing archive member falls below $\delta_{\min}$ are discarded before insertion, enforcing structural novelty. The archive serves as both the in-context demonstration pool for intra-island proposals and the candidate source for cross-island synthesis.

3.2.3 Proposal Conditioning

Every LLM call for island i at generation t is conditioned on:

$$\text{Context}_i^{(t)} = \left(\mathcal{M}, \rho_i^{(t)}, \text{Sample}\left(\mathcal{A}_i^{(t)}, n\right) \right), \quad (10)$$

where $\mathcal{M}$ grounds the proposal in domain knowledge, $\rho_i^{(t)}$ steers it with accumulated search preferences, and $\text{Sample}(\mathcal{A}_i^{(t)}, n)$ exemplifies it with the n highest-scoring programs discovered so far. This three-way conditioning closes the feedback loop introduced above: history shapes memory, memory steers proposals, and accepted proposals enrich the archive.

3.3 Topology-Aware Cross-Island Transfer

Multi-island decomposition prevents within-family saturation but cannot discover features requiring operators from multiple families simultaneously. For instance, the feature $\frac{\text{rolling_mean}(\text{sales}, 7)}{\text{lag}(\text{sales}, 3)}$ combines temporal operators ($\mathcal{P}_3$) with arithmetic operators ($\mathcal{P}_1$) neither family alone can generate it. We introduce topology-aware cross-island transfer to systematically discover such cross-family compositions, organized around three questions: *when* to transfer (saturation detection), *where* to transfer (precursor selection), and *what* to transfer (hybrid synthesis).

Definition 3 (Topology Graph). *The topology graph $\mathcal{G}^{(t)} = (V, E, \mathbf{W}^{(t)})$ is a directed weighted graph where V is the set of islands, E is the set of directed transfer edges, and $w_{j \rightarrow i}^{(t)} \in \mathbf{W}^{(t)}$ encodes the empirically observed utility of transferring knowledge from island j to i up to generation t .*

Edge weights are initialized uniformly, $w_{j \rightarrow i}^{(0)} = 1/(M - 1)$ for $j \neq i$ and updated online after each transfer event via an exponential moving average:

$$w_{j \rightarrow i}^{(t+1)} = (1 - \alpha) w_{j \rightarrow i}^{(t)} + \alpha \Delta_{\text{cross}}^{(j \rightarrow i)}(t), \quad (11)$$

where $\alpha \in (0, 1)$ is a decay coefficient and $\Delta_{\text{cross}}^{(j \rightarrow i)}(t)$ is observed fitness gain from the transfer. This makes $\mathcal{G}^{(t)}$ a continuously refined, data-driven model of cross-family complementarity updated online throughout the search.

3.3.1 When to Transfer: Saturation Detection.

An island is saturated when it has exhausted the useful information extractable under its current transformation family, so that further local search yields diminishing returns. Formally, island i is *saturated* at generation t if its expected marginal improvement from intra-island proposals falls below threshold $\varepsilon > 0$ over a sliding window of W consecutive generations:

$$\frac{1}{W} \sum_{\tau=t-W+1}^t \Delta_i(\tau) \leq \varepsilon, \quad (12)$$

where $\Delta_i(\tau) = \mathbb{E}[U_i^{\max}(\tau + 1) - U_i^{\max}(\tau)]$, and $U_i^{\max}(\tau) = \max_{p \in \Pi_i^{(\tau)}} \hat{\Phi}(p)$. Saturation detection decouples transfer from fixed schedules, triggering cross-island communication only when local search has genuinely stagnated.

3.3.2 Where to Transfer: Precursor Selection

When island i is saturated, TOPOFE selects a *precursor island* j^* , the donor whose knowledge is most likely to alleviate i 's stagnation via:

$$j^* = \arg \max_{j \neq i} w_{j \rightarrow i}^{(t)} \quad (13)$$

with ties broken at random. Over time, $\mathcal{G}^{(t)}$ identifies persistently high-utility transfer pathways (e.g., temporal $\rightarrow$ arithmetic), concentrating the search budget on productive cross-family compositions.

3.3.3 What to Transfer: Cross-Island Synthesis

Given saturated island i and precursor j^* , cross-island synthesis generates hybrid programs in the joint composition space:

$$\mathcal{P}_{i \leftarrow j^*} \subseteq \mathcal{P}_i \cup (\mathcal{P}_i \circ \mathcal{P}_{j^*}) \quad (14)$$

where $\circ$ denotes operator-level composition. The LLM is prompted with the top- m programs from $\Pi_i^{(t)}$ as target context, the top- m programs from $\Pi_{j^*}^{(t)}$ as donor context, and the task metadata $\mathcal{M}$, producing programs that integrate structural elements from $\mathcal{P}_{j^*}$, while remaining consistent with $\mathcal{P}_i$'s inductive bias. This *LLM-mediated hybrid synthesis* differs fundamentally from direct program copying, generating programs unreachable by intra-island search alone. Accepted candidates update $\Pi_i^{(t)}$ and $\mathcal{A}_i^{(t)}$, the observed gain then updates $w_{j^* \rightarrow i}^{(t)}$ via Eq.(11), ensuring future routing decisions reflect the empirical utility of this transfer pathway.

3.3.4 Unified Objective

The full TOPOFE objective jointly optimizes predictive performance, feature complementarity, stability, and computational efficiency. Let $\mathcal{A} = \bigcup_{i=1}^M \mathcal{A}_i$ denote the pooled candidate archive assembled from all island archives. The final engineered feature set $S^* \subseteq \mathcal{A}$ is obtained through redundancy-aware selection under a cardinality constraint $|S^*| \leq d'_{\max}$. The optimization objective is defined as:

$$\begin{aligned} \max_{\{\Pi_i\}, \mathcal{G}, S^* \subseteq \mathcal{A}} \quad & \mathbb{E}[\text{Perf}(f_{S^*}^*, T_{S^*}(\mathbf{X}_{\text{val}}), \mathbf{Y}_{\text{val}})] \\ & - \lambda_1 \text{Redundancy}(S^*) + \lambda_2 \text{Stability}(S^*) \\ & - \lambda_3 \text{Cost}(B) \\ \text{s.t.} \quad & |S^*| \leq d'_{\max}. \end{aligned} \quad (15)$$

Here, the predictive term evaluates the downstream generalization performance of the model trained on the engineered feature set S^* . The redundancy penalty discourages highly correlated feature programs: $\text{Redundancy}(S) = \frac{1}{|S|(|S|-1)} \sum_{p \neq p' \in S} |\text{Corr}(p(\mathbf{X}), p'(\mathbf{X}))|$, where $\text{Corr}(\cdot, \cdot)$ denotes Spearman rank correlation computed over feature outputs. This formulation excludes self-correlations and penalizes only pairwise redundancy between distinct feature programs. The stability term $\text{Stability}(S)$ measures the inverse variance of the fitness estimate $\hat{\Phi}(S)$ across cross-validation folds, encouraging feature sets whose performance remains consistent under different train-validation partitions. The computational term $\text{Cost}(B)$ represents the total oracle evaluation budget consumed during search. The hyperparameters $\lambda_1, \lambda_2, \lambda_3 \geq 0$ control the trade-off between predictive accuracy, feature diversity, robustness, and computational efficiency.

After search termination, the final feature set is constructed by greedily selecting high-scoring programs from $\mathcal{A}$ in descending validation-performance order while discarding candidates whose absolute correlation with any previously selected feature exceeds a predefined threshold τ_{red} .

3.4 Computational Complexity

Standard single-population evolutionary feature search evaluates n candidates per generation, each at cost C_{eval} , giving a total cost of $\mathcal{O}(T \cdot n \cdot C_{\text{eval}})$ over T generations. TOPOFE distributes n candidates across M islands with $\sum_i n_i = n$, incurring per-generation cost:

$$\mathcal{O}\left(\sum_{i=1}^M n_i \cdot C_{\text{eval}} + |\mathcal{E}_t| \cdot C_{\text{transfer}}\right) \quad (16)$$

where $|\mathcal{E}_t|$ is the number of active transfer edges and transfer is the cost of a single hybrid synthesis call. Under balanced allocation ($n_i \approx n/M$), this reduces to $\mathcal{O}(n \cdot C_{\text{eval}} + |\mathcal{E}_t| \cdot C_{\text{transfer}})$, matching the single-population baseline up to transfer overhead. Since transfer is saturation-triggered rather than applied at every generation, $|\mathcal{E}_t|$ remains small in practice and transfer overhead is negligible.

Parallelism. Island evaluations are mutually independent, so all M islands execute in parallel, reducing wall-clock time per generation from $\mathcal{O}(n \cdot C_{\text{eval}})$ to $\mathcal{O}(\max_i n_i \cdot C_{\text{eval}})$ and yielding an approximate k -fold speedup under balanced partitioning.

Algorithm 1: TOPOFE

Input: $\mathcal{D}, \mathcal{M}, \{\mathcal{P}_i\}_{i=1}^M$, budget B , window W , threshold ε , decay α

Output: S^*

for $i = 1, \dots, K$ **do**

$\Pi_i^{(0)} \leftarrow \text{LLM-Seed}(\mathcal{P}_i, \mathcal{M}); \mathcal{A}_i^{(0)} \leftarrow \emptyset; \rho_i^{(0)} \leftarrow \emptyset;$

Initialize $\mathcal{G}^{(0)}$ with $w_{j \rightarrow i}^{(0)} = 1/(M-1)$ for all $j \neq i; t \leftarrow 0, b \leftarrow 0;$

while $b < B$ **do**

for $i = 1, \dots, M$ **do in parallel**

$p' \leftarrow \text{LLM-Mutate/Crossover}(\mathcal{A}_i^{(t)}, \mathcal{M}, \rho_i^{(t)});$

 Evaluate $\hat{\Phi}(p'); b \leftarrow b + 1;$

 Update $\Pi_i^{(t+1)}, \mathcal{A}_i^{(t+1)}, \mathcal{H}_i^{(t+1)}, \rho_i^{(t+1)}$ via Eqs. 8, 9;

for $i = 1, \dots, M$ **do**

if Saturated(i, W, ε) **then**

$j^* \leftarrow \arg \max_{j \neq i} w_{j \rightarrow i}^{(t)};$

$p' \leftarrow \text{LLM-HybridSynth}(\mathcal{A}_i^{(t)}, \mathcal{A}_{j^*}^{(t)}, \mathcal{M});$

 Evaluate $\hat{\Phi}(p'); b \leftarrow b + 1;$

 Update $\Pi_i^{(t+1)}, \mathcal{A}_i^{(t+1)};$

$w_{j^* \rightarrow i}^{(t+1)} \leftarrow (1 - \alpha)w_{j^* \rightarrow i}^{(t)} + \alpha \Delta_{\text{cross}}^{(j^* \rightarrow i)}(t);$

$t \leftarrow t + 1;$

return $S^* \leftarrow \text{GreedySelect}(\bigcup_{i=1}^M \mathcal{A}_i^{(t)}, d'_{\max})$

Sample Efficiency. Saturation-triggered transfer concentrates oracle evaluations on high-utility regions of $\mathcal{P}$, reducing the number of generations T required to converge relative to single-population search. TOPOFE thus achieves comparable asymptotic complexity to the baseline while improving both wall-clock efficiency through parallelism and convergence speed through principled cross-family exploration.

4 Experiments

We conduct experiments to address the following research questions. **RQ1:** Can TOPOFE outperform existing state-of-the-art AutoFE methods across diverse tabular learning tasks and dataset characteristics? (§5.1) **RQ2:** Is TOPOFE’s performance robust to the choice of LLM backbone across models of varying capability? (§5.2) **RQ3:** Are the feature programs discovered by TOPOFE transferable across architecturally distinct downstream predictors beyond the search-time evaluation model? (§5.3) **RQ4:** Does TOPOFE produce feature sets that are more diverse, low-redundancy, and informationally complementary than competing methods? (§5.4) **RQ5:** Does the adaptive topology graph acquire meaningful task-specific knowledge about cross-family transfer utility during search? (§5.5) **RQ6:** How each component of TOPOFE contributes to the its overall performance? (§5.6)

Datasets We evaluate TOPOFE on 29 public tabular datasets comprising 19 classification and 10 regression tasks, sourced from the UCI Machine Learning Repository [Asuncion et al., 2007]¹, Kaggle², and OpenML [Vanschoren et al., 2014]³, following the dataset selection protocol of prior work [Hollmann et al., Han et al., 2024, Abhyankar et al., 2025]. The benchmark spans diverse prediction tasks, dataset scales ($n_{inst} \in [315, 581012]$), feature dimensionalities ($n_{feat} \in [4, 279]$), and heterogeneous feature types including numerical, categorical, and temporal variables. Dataset statistics are reported in Tables 1 and 2. Each dataset is accompanied by structured metadata $\mathcal{M}$ comprising task-level descriptions and per-feature semantic annotations used to condition LLM generation. All datasets are partitioned using a fixed 80/20 train/test split, where the training partition is used exclusively for feature search, cross-validation, and model fitting, and the test partition is held out for final unbiased evaluation. All experiments are repeated over five runs with different random seeds and data splits. Results are reported as mean $\pm$ standard deviation.

¹<https://archive.ics.uci.edu/datasets>

²<https://www.kaggle.com/datasets>

³<https://www.openml.org/>

Baselines We compare TOPOFE against six SOTA AutoFE methods spanning three categories: (i) *classical methods*: OpenFE [Zhang et al., 2023a] and AutoFeat [Horn et al., 2019], (ii) *LLM-based methods*: CAAFE [Hollmann et al., 2023a], FeatLLM [Han et al., 2024], and OCTree [Nam et al., 2024], and (iii) *LLM-guided evolutionary search*: LLM-FE [Abhyankar et al., 2025], the most closely related baseline sharing the multi-population evolutionary architecture. All LLM-based methods use the same backbone model and generation settings as TOPOFE to ensure a controlled comparison.

Evaluation Protocol Classification performance is measured by accuracy ($\uparrow$) and regression performance by Root Mean Square Error ($\downarrow$). Feature evaluation follows a two-stage procedure: given a candidate feature set S , we first execute each program $p \in S$ on the training data to construct the augmented representation $\mathcal{T}_S(\mathbf{X}_{\text{tr}})$, then train a downstream predictor f_S^* on the augmented training set and evaluate the fitness signal $\hat{\Phi}(S)$ on the held-out validation fold via 5-fold stratified cross-validation as defined in Eq. (6). For fair comparison across all methods, XGBoost is used as the default downstream predictor with identical train/validation protocols, evaluation budgets, and model configurations unless explicitly stated otherwise.

Implementation Details. All LLM-based methods are implemented with three backbone models: Qwen3-8B [Yang et al., 2025a], Qwen2.5-Coder [Hui et al., 2024], and GPT-4o-mini [Hurst et al., 2024]. Unless otherwise specified, we use a sampling temperature of $\tau = 0.8$ for all LLM generation calls. For TOPOFE, we instantiate $M = 5$ family-specialised islands corresponding to the five canonical transformation families defined in §3.1: arithmetic interactions ($\mathcal{P}_1$), statistical aggregates ($\mathcal{P}_2$), temporal features ($\mathcal{P}_3$), relational encodings ($\mathcal{P}_4$), and nonlinear univariate maps ($\mathcal{P}_5$). At each generation, each LLM call produces $b = 3$ candidate feature programs written as executable Python functions.

Each LLM prompt comprises four components: (i) task-level metadata $\mathcal{M}^{\text{task}}$ summarizing the prediction objective and target variable, (ii) feature-level metadata $\mathcal{M}^{\text{feat}}$ comprising feature names, value types, and one-sentence semantic descriptions, with representative category values included for categorical variables, (iii) in-context demonstrations drawn from the island’s elite archive $\mathcal{A}_i^{(t)}$, providing high-quality previously discovered programs as exemplars for mutation, crossover, or hybrid synthesis, and (iv) a strict output format specification requiring a syntactically valid Python feature-transformation function. A small number of serialized training instances with ground-truth labels are additionally included as task context, following [Dinh et al., 2022, Han et al., 2024]. All generated programs undergo systematic validation comprising syntax checking, type consistency verification, numerical-safety filtering (e.g., division-by-zero and overflow guards), and metadata-consistency checks before fitness evaluation.

For the PAM component, the sliding history window is set to $W = 5$ generations and the prompt-memory string is updated after every generation via an LLM summarization call conditioned on $\mathcal{H}_i^+(t)$ and $\mathcal{H}_i^-(t)$. The elite archive capacity is $|\mathcal{A}|_{\max} = 20$ programs per island, with structural novelty enforced via a normalized tree-edit distance threshold of $\delta_{\min} = 0.1$. For the topology graph, the exponential moving average decay coefficient is $\alpha = 0.3$, and the saturation threshold is $\varepsilon = 0.01$ over a window of $W = 5$ consecutive generations. The redundancy penalty coefficient is $\lambda_1 = 0.3$, stability weight $\lambda_2 = 0.1$, and cost weight $\lambda_3 = 0.1$ in the unified objective (Eq. 15). After search termination, the final feature set is assembled by pooling elite programs from all island archives and selecting at most 5 non-redundant programs via greedy validation-score-ranked selection, discarding candidates whose absolute Spearman correlation with any already-selected feature exceeds $\tau_{\text{red}} = 0.9$.

5 Results and Analysis

5.1 Main Results

Tables 1 and 2 report classification accuracy and regression RMSE respectively across all datasets and methods under a unified evaluation protocol, where all methods are paired with the same XGBoost learner and all LLM-based approaches share an identical Qwen3-8B backbone.

Classification. As shown in Table 1, TOPOFE achieves the best performance on 15 of 19 datasets while maintaining consistently competitive, and often lower, variance across runs, demonstrating a favorable balance between predictive performance and search stability. Gains scale systematically with dataset complexity. On near-saturated, low-dimensional datasets (adult, tic-tac-toe), classical methods remain competitive and TOPOFE’s margins are marginal, confirming that structured multi-family search does not introduce unnecessary overhead when enumeration suffices. Gains become substantial on medium-complexity datasets requiring compositional interactions (e.g., heart, balance-scale, credit-g, eucalyptus) where TOPOFE improves over the next-best through saturation-triggered cross-family transfer that discovers interaction features inaccessible to any single-family grammar. On large-scale datasets (diabetes), TOPOFE achieves the best performance while LLM-FE collapses. The gap between LLM-FE and TOPOFE is largest on

Table 1: Performance of XGBoost on classification datasets with different AutoFE methods, measured by accuracy ($\uparrow$). Results are reported in mean $\pm$ std over five splits. Best in **bold**, second-best underlined. Qwen3-8B is used as the backbone for all LLM-based methods. n_{inst} is the number of instances and n_{feat} is the number of features. ‘Base’ represents the performance of XGBoost w/o FE.

Dataset	n_{inst}	n_{feat}	Base	Classical AutoFE		LLM-based FE			LLM+Evolutionary FE	
				AutoFeat	OpenFE	CAAFE	FeatLLM	OTree	LLMFE	TopoFE
adult	48842	14	0.8611 $\pm$ 0.0165	0.9265 $\pm$ 0.0011	<u>0.9267</u> $\pm$ 0.0013	0.8784 $\pm$ 0.0052	0.8671 $\pm$ 0.0298	0.8668 $\pm$ 0.0104	0.8708 $\pm$ 0.0011	0.9268 $\pm$ 0.0025
arrhythmia	452	279	0.7053 $\pm$ 0.0186	0.6702 $\pm$ 0.0371	0.7344 $\pm$ 0.0362	0.7143 $\pm$ 0.0127	0.6561 $\pm$ 0.0296	0.6560 $\pm$ 0.0335	0.6566 $\pm$ 0.0129	<u>0.7144</u> $\pm$ 0.0203
balance-scale	625	4	0.8320 $\pm$ 0.0233	0.8848 $\pm$ 0.0079	<u>0.8560</u> $\pm$ 0.0036	0.8320 $\pm$ 0.0054	0.8252 $\pm$ 0.0315	0.7822 $\pm$ 0.0241	0.8780 $\pm$ 0.0093	0.9465 $\pm$ 0.0046
bank	45211	16	0.8991 $\pm$ 0.0118	0.9020 $\pm$ 0.0019	<u>0.9297</u> $\pm$ 0.0015	0.9091 $\pm$ 0.0031	0.9097 $\pm$ 0.0030	0.8985 $\pm$ 0.0228	0.9054 $\pm$ 0.0075	0.9333 $\pm$ 0.0182
breast-w	699	9	0.9500 $\pm$ 0.0096	0.9728 $\pm$ 0.0054	0.9906 $\pm$ 0.0005	0.9429 $\pm$ 0.0043	<u>0.9913</u> $\pm$ 0.0252	0.9597 $\pm$ 0.0130	0.9607 $\pm$ 0.0036	0.9942 $\pm$ 0.0014
blood	748	4	0.6733 $\pm$ 0.0018	0.6805 $\pm$ 0.0056	0.6924 $\pm$ 0.0167	<u>0.7733</u> $\pm$ 0.0100	0.7749 $\pm$ 0.0051	0.7246 $\pm$ 0.0312	0.7208 $\pm$ 0.0209	0.7440 $\pm$ 0.0129
car	1728	6	0.9671 $\pm$ 0.0027	0.9872 $\pm$ 0.0030	0.9913 $\pm$ 0.0062	0.9819 $\pm$ 0.0080	0.8150 $\pm$ 0.0087	0.9783 $\pm$ 0.0164	0.9734 $\pm$ 0.0095	0.9971 $\pm$ 0.0051
diabetes	253680	21	0.8491 $\pm$ 0.0068	0.8012 $\pm$ 0.0143	<u>0.8483</u> $\pm$ 0.0005	0.8491 $\pm$ 0.0096	0.8497 $\pm$ 0.0040	0.8305 $\pm$ 0.0069	0.8298 $\pm$ 0.0054	0.8524 $\pm$ 0.0021
cmc	1473	9	0.5051 $\pm$ 0.0075	0.5261 $\pm$ 0.0108	<u>0.5301</u> $\pm$ 0.0112	0.5051 $\pm$ 0.0115	0.4800 $\pm$ 0.0020	0.5111 $\pm$ 0.0334	0.5170 $\pm$ 0.0194	0.5577 $\pm$ 0.0078
communities	1994	103	0.6943 $\pm$ 0.0239	0.6991 $\pm$ 0.0075	0.6936 $\pm$ 0.0041	<u>0.6992</u> $\pm$ 0.0079	0.5938 $\pm$ 0.0013	0.6917 $\pm$ 0.0110	0.6947 $\pm$ 0.0105	0.7053 $\pm$ 0.0043
covtype	581012	54	0.8652 $\pm$ 0.0120	0.8668 $\pm$ 0.0153	0.8684 $\pm$ 0.0006	0.8652 $\pm$ 0.0076	0.8585 $\pm$ 0.0081	0.8332 $\pm$ 0.0115	0.8773 $\pm$ 0.0087	0.8774 $\pm$ 0.0038
credit-g	1000	20	0.7000 $\pm$ 0.0237	0.7629 $\pm$ 0.0019	<u>0.7725</u> $\pm$ 0.0221	0.7000 $\pm$ 0.0107	0.7124 $\pm$ 0.0076	0.7373 $\pm$ 0.0020	0.7500 $\pm$ 0.0147	0.7878 $\pm$ 0.0066
eucalyptus	736	19	0.6724 $\pm$ 0.0151	0.6643 $\pm$ 0.0213	<u>0.6766</u> $\pm$ 0.0180	0.6284 $\pm$ 0.0093	0.6191 $\pm$ 0.0062	0.6416 $\pm$ 0.0267	0.6514 $\pm$ 0.0080	0.6990 $\pm$ 0.0094
heart	918	11	0.8696 $\pm$ 0.0190	0.8923 $\pm$ 0.0012	<u>0.9199</u> $\pm$ 0.0043	0.8696 $\pm$ 0.0093	0.8789 $\pm$ 0.0160	0.8217 $\pm$ 0.0093	0.8352 $\pm$ 0.0163	0.9346 $\pm$ 0.0015
jungle-chess	44819	6	0.8636 $\pm$ 0.0019	0.8446 $\pm$ 0.0202	<u>0.8707</u> $\pm$ 0.0020	0.8636 $\pm$ 0.0088	0.5915 $\pm$ 0.0252	0.7796 $\pm$ 0.0096	0.8693 $\pm$ 0.0146	0.8858 $\pm$ 0.0104
myocardial	686	92	0.7246 $\pm$ 0.0087	0.7232 $\pm$ 0.0138	0.7217 $\pm$ 0.0350	0.7536 $\pm$ 0.0114	0.7824 $\pm$ 0.0056	0.7827 $\pm$ 0.0175	0.7847 $\pm$ 0.0036	0.7878 $\pm$ 0.0066
pcl	1109	21	0.9214 $\pm$ 0.0135	0.9313 $\pm$ 0.0153	0.9367 $\pm$ 0.0254	0.9414 $\pm$ 0.0101	0.9605 $\pm$ 0.0295	0.9302 $\pm$ 0.0186	0.9312 $\pm$ 0.0102	<u>0.9456</u> $\pm$ 0.0066
tic-tac-toe	958	9	0.9896 $\pm$ 0.0022	0.9923 $\pm$ 0.0012	0.9999 $\pm$ 0.0001	0.9896 $\pm$ 0.0134	0.6582 $\pm$ 0.0080	0.9886 $\pm$ 0.0058	0.9896 $\pm$ 0.0138	<u>0.9993</u> $\pm$ 0.0220
vehicle	846	18	0.7617 $\pm$ 0.0019	0.7342 $\pm$ 0.0143	0.7541 $\pm$ 0.0067	<u>0.7647</u> $\pm$ 0.0085	0.7519 $\pm$ 0.0106	0.7416 $\pm$ 0.0018	0.7574 $\pm$ 0.0033	0.7781 $\pm$ 0.0073

Table 2: Performance of XGBoost on regression datasets with different AutoFE methods, measured by RMSE $\downarrow$ (FeatLLM and CAAFE are designed specific for classification tasks only).

Dataset	n_{inst}	n_{feat}	Base	Classical AutoFE		LLM-based FE		LLM+Evolutionary FE	
				AutoFeat	OpenFE	OTree	LLMFE	TopoFE	
airfoil_self_noise	1503	6	1.6701 $\pm$ 0.0514	1.6265 $\pm$ 0.1701	1.6143 $\pm$ 0.0232	1.6701 $\pm$ 0.0763	<u>1.5415</u> $\pm$ 0.0162	1.3000 $\pm$ 0.0123	
bike	17389	12	4.3494 $\pm$ 0.0745	4.4713 $\pm$ 0.1830	<u>4.0471</u> $\pm$ 0.0576	4.2336 $\pm$ 0.1273	4.3494 $\pm$ 0.2363	3.0210 $\pm$ 0.0202	
cpu	8192	10	3.0352 $\pm$ 0.1953	3.0670 $\pm$ 0.1306	<u>2.4807</u> $\pm$ 0.0904	3.0087 $\pm$ 0.1007	2.9035 $\pm$ 0.1042	2.4719 $\pm$ 0.0351	
crab	3893	8	2.4700 $\pm$ 0.1039	2.4073 $\pm$ 0.0741	<u>2.4807</u> $\pm$ 0.1341	2.4222 $\pm$ 0.0254	<u>2.3489</u> $\pm$ 0.1800	2.2585 $\pm$ 0.0512	
diamond	53940	9	5.8112 $\pm$ 0.0522	<u>5.5654</u> $\pm$ 0.1203	5.9627 $\pm$ 0.0573	5.8208 $\pm$ 0.0609	5.6901 $\pm$ 0.2363	5.4711 $\pm$ 0.2299	
forest-fires	517	13	0.1751 $\pm$ 0.1939	<u>0.1651</u> $\pm$ 0.0152	0.2121 $\pm$ 0.1313	0.1740 $\pm$ 0.1956	0.1676 $\pm$ 0.0162	0.1611 $\pm$ 0.0221	
housing	20640	9	4.8168 $\pm$ 0.2270	5.1293 $\pm$ 0.1553	4.8213 $\pm$ 0.1457	<u>4.8148</u> $\pm$ 0.1256	4.8597 $\pm$ 0.2111	4.6714 $\pm$ 0.1723	
insurance	1338	7	5.2817 $\pm$ 0.2510	5.1103 $\pm$ 0.1670	4.8053 $\pm$ 0.2061	4.9810 $\pm$ 0.2513	5.0812 $\pm$ 0.3133	<u>4.8843</u> $\pm$ 0.1802	
plasma	315	13	2.3577 $\pm$ 0.0455	2.2840 $\pm$ 0.1397	2.2409 $\pm$ 0.1554	<u>2.3677</u> $\pm$ 0.1007	2.2835 $\pm$ 0.2358	2.1950 $\pm$ 0.0451	
wine	2554	11	0.6911 $\pm$ 0.0684	0.6847 $\pm$ 0.0252	<u>0.6557</u> $\pm$ 0.0455	0.6911 $\pm$ 0.0822	0.6619 $\pm$ 0.0719	0.5936 $\pm$ 0.0098	

datasets requiring cross-family compositions (e.g., balance-scale, heart, cmc), directly isolating heterogeneous family specialization, learned transfer topology, and adaptive prompt memory as the decisive architectural advantages.

Regression As shown in Table 2, TOPOFE attains the lowest RMSE on 9 regression datasets, with improvements scaling consistently with cross-variable interaction richness: largest on interaction-driven datasets (bike: 28.7% reduction over OTree, airfoil: 15.7% over LLM-FE) and narrowest where a single transformation family suffices (cpu, forest-fires). LLM-FE’s performance gap relative to TOPOFE widens with dataset complexity, consistent with its single-population design collapsing onto a dominant transformation family. Across both task types, TOPOFE’s lower standard deviations confirm that per-island Prompt Adaptation Memory and schema-constrained synthesis jointly reduce the instability inherent in unconstrained LLM generation.

5.2 Generalizability

To evaluate whether TOPOFE’s effectiveness is contingent on a specific LLM backbone, we instantiate it with three models spanning a representative capability range: Qwen2.5-Coder-7B [Hui et al., 2024], a code-specialised open-source model; Qwen3-8B [Yang et al., 2025a], a general-purpose open-source model; and GPT-4o-mini [Hurst et al., 2024], a stronger proprietary model. All other system components remain identical across configurations.

Table 3 establishes two findings. First, TOPOFE achieves consistently strong performance across all three backbones on both task types. The small performance variation across backbones differing in capability and training methodology confirms that TOPOFE’s core gains are driven primarily by its architectural mechanisms and slightly affected by the generative capability of LLMs. The evolutionary framework effectively compensates for weaker proposal quality through iterative fitness-guided selection. Second, classification accuracy improves monotonically with backbone capability, consistent with stronger models producing more schema-compliant programs on the first attempt and reducing wasted evaluation budget. These results confirm that TOPOFE is backbone-agnostic in a practically meaningful sense, deployable across diverse resource constraints without architectural modification.

Table 3: Average TOPOFE performance across all classification and regression datasets under three LLM backbones.

Task	QwenCode-7B	Qwen3-8B	GPT-4o-mini
Classification (Accuracy $\uparrow$)	0.8371	0.8403	0.8468
Regression (RMSE $\downarrow$)	2.6790	2.6928	2.6342

Table 4: Average downstream performance using the same engineered feature set generated by TOPOFE with XGBoost as the default predictor.

Task	XGB	CatBoost	MLP	TabPFN
Classification (Accuracy $\uparrow$)	0.837	0.849	0.816	0.873
Regression (RMSE $\downarrow$)	2.679	2.624	2.993	2.591

5.3 Feature Transferability

A critical property of any feature engineering method is that discovered transformations encode dataset-intrinsic structure rather than artifacts of the search-time model. To evaluate this, we fix the feature programs evolved using XGBoost and apply the resulting augmented feature matrix without regenerating, reselecting, or modifying, to CatBoost [Prokhorenkova et al., 2018], MLP [tau, 2017], and TabPFN [Hollmann et al., 2023b], using Qwen2.5-Coder-7B as the backbone.

Table 4 yields three findings. First, CatBoost achieves 0.849 (+1.43%) and RMSE 2.624 (−2.05%), confirming that TOPOFE’s features transfer without degradation across gradient-boosting variants with fundamentally different algorithmic mechanisms and are not overfit to XGBoost’s split-finding algorithm. Second, MLP underperforms, attributable to the well-documented sensitivity of neural network optimizers to unnormalized skewed feature distributions [Ioffe and Szegedy, 2015] instead of a failure of transferability. Standard normalization would be expected to substantially close this gap. Third, TabPFN achieves the best performance on both tasks, surpassing even the search-time XGBoost oracle by 4.30% and 3.29%. This super-transfer result demonstrates TOPOFE’s programs encode richer structure than any single surrogate can fully exploit, and TabPFN’s meta-learned attention mechanism is particularly well-suited to exploit the orthogonal arithmetic, temporal, rank-based, and aggregation representations that TOPOFE’s family-specialized islands produce. These results confirm that transferability is a stable, task-agnostic property of TOPOFE’s feature programs, with improvement magnitude scaling with each predictor’s capacity to exploit structured low-redundancy representations.

5.4 Feature Diversity

To evaluate the structural quality and informational richness of the feature sets produced by each method, we define two geometrically complementary diversity metrics. The *Mean Pairwise Output Correlation* (MPOC) measures local pairwise redundancy as the average absolute Spearman rank correlation between all pairs of output vectors in the accepted engineered feature set Φ :

$$\text{MPOC}(\Phi) = \frac{2}{|\Phi|(|\Phi| - 1)} \sum_{i < j} |\rho_{\text{sp}}(\phi_i(\mathbf{X}), \phi_j(\mathbf{X}))|, \quad (17)$$

where ρ_{sp} denotes the Spearman rank correlation evaluated on the training set. Lower MPOC indicates that the accepted features carry less redundant information, with output vectors spanning a larger subspace of $\mathbb{R}^n$ instead of collapsing into a low-rank correlated block.

Effective Rank (EffRank) measures global dimensional coverage from the normalized singular-value spectrum of the engineered feature matrix:

$$\text{EffRank}(\Phi) = \exp \left(- \sum_{k=1}^r \tilde{\sigma}_k \log \tilde{\sigma}_k \right), \quad \tilde{\sigma}_k = \frac{\sigma_k}{\sum_{j=1}^r \sigma_j} \quad (18)$$

where $\{\sigma_k\}_{k=1}^r$ are the singular values. Higher EffRank indicates broader coverage of independent feature directions. We report EffRank normalized by $|\Phi|$ for comparability across methods with different numbers of accepted features.

Table 5 reveals that TOPOFE achieves the lowest MPOC and highest EffRank across both classification and regression, establishing joint optimality in redundancy suppression and subspace coverage that no competing method achieves.

Table 5: Average MPOC and Effective Rank over all classification and regression datasets across all methods.

Method	Classification		Regression	
	MPOC ↓	EffRank↑	MPOC↓	EffRank↑
OpenFE	0.421	0.480	0.378	0.448
AutoFeat	0.513	0.330	0.486	0.309
FeatLLM	0.581	0.394	0.462	0.389
CAAFE	0.479	0.567	0.382	0.412
LLMFE	0.309	0.699	0.320	0.631
TOPOFE	0.261	0.832	0.232	0.680

The per-method profiles expose mechanistically distinct failure modes: AutoFeat attains the highest MPOC and lowest EffRank because polynomial enumeration generates strongly monotone-related features that collapse the singular value spectrum into few dominant directions. FeatLLM similarly suffers high MPOC and low EffRank through thematic clustering of stateless LLM proposals that are semantically coherent but informatively redundant. CAAFE and OpenFE occupy an intermediate regime where iterative feedback and feature boosting partially suppress redundancy but remain confined to a single operator grammar, precluding high EffRank. LLM-FE improves substantially yet is ultimately constrained by its homogeneous, grammar-agnostic islands that concentrate the feature matrix within a single undifferentiated operator subspace. TOPOFE’s joint advantage is explained by two complementary mechanisms absent from all prior work: an explicit within-search redundancy filter that prevents correlated clusters from accumulating (driving MPOC down), and heterogeneous family schema combined with cross-family hybrid synthesis that import structurally orthogonal templates from precursor islands into the target grammar (driving EffRank up), together constituting the geometric signature of TOPOFE’s architectural superiority.

5.5 Topology Graph Specialization

To evaluate whether TOPOFE’s topology graph acquires task-specific knowledge about cross-family transfer utility over the course of search, we define the *Topology Graph Specialization Score* (TGSS). Let $W^{(t)} \in \mathbb{R}^{M \times M}$ denote the transfer-weight matrix of the topology graph at generation t , where each off-diagonal entry $w_{j \rightarrow i}^{(t)} \in [-1, 1]$ encodes the learned utility of island j as a precursor for saturated island i and is updated according to Eq. 11. TGSS is defined as the empirical variance of the off-diagonal transfer weights:

$$\text{TGSS}(t) = \frac{1}{M(M-1)} \sum_{\substack{j,i=1 \\ j \neq i}}^M \left(w_{j \rightarrow i}^{(t)} - \bar{w}^{(t)} \right)^2 \quad (19)$$

where the mean off-diagonal weight is defined as

$$\bar{w}^{(t)} = \frac{1}{M(M-1)} \sum_{\substack{j,i=1 \\ j \neq i}}^M w_{j \rightarrow i}^{(t)}. \quad (20)$$

TGSS quantifies the structural divergence of the weight matrix from its uniform initialization: $\text{TGSS}(0) = 0$ exactly when all weights are equal, and increases as the graph develops differentiated preferences over island pairs. A high TGSS indicates that the topology graph has identified strongly productive and strongly unproductive transfer directions specific to the current task, concentrating transfer routing accordingly. A near-zero TGSS indicates that all island pairs yield approximately equal transfer utility and the graph remains close to its uninformative prior. TGSS directly measures whether the adaptive topology mechanism is acquiring task-specific knowledge that distinguishes TOPOFE from fixed or random transfer alternatives.

Figure 2 reports TGSS trajectories across all classification and regression datasets over 10 generations. All datasets begin at $\text{TGSS}(0) = 0$ and increase monotonically, confirming that the topology graph consistently specializes away from its uniform prior as transfer events accumulate — a necessary precondition for adaptive routing to confer any advantage over random precursor selection. Three patterns emerge from the joint analysis. First, TGSS exhibits substantial inter-dataset variance at convergence, spanning $[0.00, 0.22]$ for classification and $[0.04, 0.24]$ for regression, confirming that the degree of learnable cross-family utility structure is genuinely dataset-dependent instead of a fixed algorithmic property. This validates TOPOFE’s adaptive design: a fixed topology would be suboptimal for all datasets, while the learned graph adapts to the specific inter-family utility structure of each task. Second, datasets with near-zero TGSS

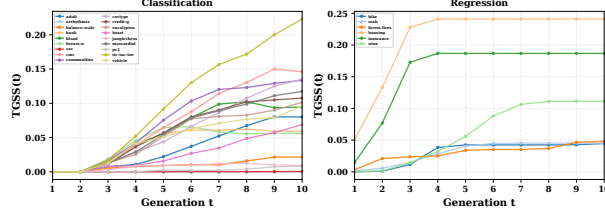


Figure 2: TGSS trajectories across all classification (left) and regression (right) datasets over 10 generations.

throughout, notably tic-tac-toe and blood dataset (classification) are precisely those where TOPOFE’s performance advantage over single-family baselines is smallest, establishing TGSS as a practically useful predictive indicator of when cross-family transfer is likely to be beneficial. Conversely, vehicle (classification, $TGSS \approx 0.22$) and forest-fires (regression, $TGSS \approx 0.24$) achieve the highest specialization, consistent with their heterogeneous feature interactions across multiple operator families that produce strongly differentiated island pair utilities. Third, regression datasets exhibit faster early plateau behavior. Most curves flatten after generation 3–4, while classification curves continue growing through generation 10. This asymmetry reflects that regression fitness signals are smoother and more stable, enabling the bandit-style update rule to reach confident weight estimates in fewer transfer trials, while classification tasks present noisier utility landscapes that require sustained transfer sampling to resolve inter-family utility differences. This observation motivates task-type-specific transfer budget scheduling as a direction for future work.

5.6 Ablation Study

To quantify the individual contribution of each architectural component in TOPOFE, we conduct a systematic ablation study by progressively removing or modifying key mechanisms while holding all other components fixed. We evaluate ten variants organised into four groups corresponding to the main architectural decisions: island decomposition, migration strategy, transfer mechanism, and saturation detection.

Island decomposition: (i) *Single Pool* removes island decomposition and migration, reducing TOPOFE to single-population search; (ii) *Random Island* partitions the population into islands without family-aware assignment. **Migration strategy:** (iii) *w/o Inter-island Exchange* retains family decomposition but disables cross-island migration; (iv) *Static Migration* replaces saturation-triggered transfer with fixed-interval migration; (v) *Stochastic Migration* selects precursor islands uniformly at random. **Transfer mechanism:** (vi) *Direct Transfer* copies programs from the precursor island without hybrid synthesis; (vii) *Unguided Hybridisation* enables hybrid synthesis with randomly selected precursors, disabling topology guidance; (viii) *w/o Hybridisation* preserves topology-guided precursor selection but removes LLM-mediated synthesis. **Saturation detection:** (ix) *Correlation-Only* replaces the composite saturation criterion with a single output-correlation signal; (x) *Multi-signal Saturation* restores full multi-signal adaptive saturation detection without other full-system components.

Figure 3 summarizes performance degradation across all variants on both task types. The largest drop occurs for *Single Pool*, confirming that unstructured single-population search is the primary bottleneck: without island decomposition, search collapses toward dominant transformation patterns, causing premature convergence and reduced diversity. The improvement from *Random Island* to *w/o Inter-island Exchange* further demonstrates that family-aware decomposition contributes meaningful inductive structure beyond simple partitioning, improving local optimization efficiency by conditioning LLM proposals on a homogeneous operator context. Migration ablations show that transfer effectiveness depends critically on both timing and precursor quality. *Static Migration* wastes transfer budget when local search remains productive, while *Stochastic Migration* degrades further by introducing structurally unrelated transformations that disrupt locally adapted populations. Transfer mechanism ablations confirm the necessity of compositional synthesis: *Direct Transfer* yields limited gains as copied programs remain constrained by their source family’s inductive bias. LLM-mediated synthesis overcomes this by generating programs spanning multiple families that are unreachable by intra-island search alone. *Unguided Hybridization* remains suboptimal despite enabling synthesis, demonstrating that topology guidance and hybrid synthesis are synergistic — learned precursor selection ensures hybridization targets complementary family pairs, maximizing the probability of introducing non-redundant predictive structure. Saturation detection ablations confirm that multi-signal stagnation detection is essential. *Correlation-Only Saturation* underperforms because output correlation alone cannot distinguish genuine stagnation from transient redundancy — an island may stagnate in performance while maintaining low correlation, or exhibit high correlation from isolated redundant proposals while remaining broadly productive. The multi-signal composite criterion correctly discriminates these cases, triggering transfer only when local search is genuinely exhausted across all dimensions of diversity and performance.

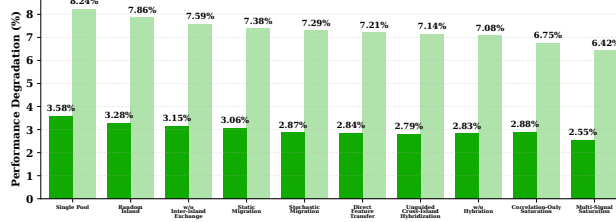


Figure 3: Ablation study of TOPOFE. Each bar reports the performance degradation relative to the full TOPOFE. Results are averaged across all datasets.

6 Related Work

6.1 Automatic Feature Engineering

Automated feature engineering (AutoFE) has evolved through three broad paradigms. *Search-based methods* treat feature construction as a combinatorial optimization problem, through genetic programming [Olson and Moore, 2016], exhaustive [Horn et al., 2019] and greedy search [Zhang et al., 2023a]. These methods are computationally expensive and domain-agnostic without semantic guidance. *LLM-guided methods* use pretrained LMs to propose features [Han et al., 2024, Ko et al., 2025, Nam et al., 2024] from task descriptions and dataset metadata [Hollmann et al., 2023a]. Recent extensions apply Tree-of-Thought reasoning [Zhang et al., 2025], multi-agent frameworks [Ouyang et al., 2025], and ReAct-based agents [Burghardt et al., 2026] to guide feature discovery. However, these methods rely on prompting, leading to repeated proposals, no dataset-specific adaptation, and no cross-family composition. *LLM-evolutionary methods* combine LLM-based generation with evolutionary search [Abhyankar et al., 2025, Gong et al., 2025, Batista, 2025]. While these methods improve over static prompting, they operate over a single population without structural decomposition of the program space, precluding cross-family hybridization and leaving the search vulnerable to premature convergence within a dominant transformation family. TOPOFE addresses these limitations directly through multi-island decomposition, prompt adaptation memory, and topology-guided cross-family transfer.

6.2 LLMs for Tabular Data Learning

Recent advances in tabular learning have converged toward two main paradigms. *Tabular foundation models* pursue cross-table transferability through large-scale pretraining, ranging from Bayesian inference approximation via Prior-Data Fitted Networks [Hollmann et al., 2023b, 2025] and architecture-level scaling for in-context learning [Gardner et al., 2024, Arazi et al., 2026, Qu et al., 2025], to retrieval-augmented inference without task-specific fine-tuning [Ma et al., 2025]. *LLM-based tabular methods* repurpose pretrained LMs for few-shot prediction [Hegselmann et al., 2023, Dinh et al., 2022], semantic generalization via textual metadata [Ye et al., 2024a, Kim et al., 2024, Yan et al.], and auxiliary tasks including data augmentation [Zhang et al., 2023b, Borisov et al.], data cleaning [Bendinelli et al.], and feature generation [Hollmann et al., 2023a]. Despite these advances, existing approaches largely rely on per-task adaptation and lack principled search mechanisms over the feature space, which TOPOFE is specifically designed to provide.

6.3 LLM-guided Evolutionary Search

A growing body of work integrates LLMs as evolutionary operators across prompt optimization [Guo et al., 2023, Fernando et al., Suzgun et al., 2026, Agrawal et al.], program and algorithm discovery [Šurina et al., Assumpção et al., 2025, Novikov et al., 2025, Sharma, Lange et al., 2025], and scientific discovery [Shojaee et al., Yang et al., 2025b, Chen et al., 2026, Abhyankar et al., 2026]. These systems vary primarily in population dynamics, employing MAP-Elites [Novikov et al., 2025], island-based evolution [Assumpção et al., 2025], Pareto frontier optimization [Agrawal et al.], and diversity-driven selection [Sharma], with more advanced frameworks incorporating co-evolution and meta-reflection [Liu et al., Ye et al., 2024b] or iterative agent evolution [Yuan et al., 2025, Shang et al.]. Despite their effectiveness, existing methods treat the search space as structurally uniform and do not incorporate semantic decomposition, task-adaptive proposal conditioning, or saturation-triggered transfer, all of which are central to TOPOFE.

7 Conclusion

We present TOPOFE, a LLM-guided framework for automated feature engineering that models search as graph-structured multi-island evolution over heterogeneous transformation families, addressing three fundamental limitations of existing methods: homogeneous search dynamics, stateless LLM querying, and rigid migration topology. Family-specialised islands preserve transformation diversity by construction; a learned directed topology graph routes cross-family transfer toward empirically productive island pairs; and saturation-triggered LLM-mediated hybrid synthesis realises transfer as compositional program generation precisely when local search is exhausted. Across 19 classification and 10 regression benchmarks, TOPOFE consistently outperforms all baselines, with the largest gains on datasets requiring cross-family feature interactions. Three purpose-built metrics confirm the mechanistic basis: MPOC and EffRank demonstrate lower-redundancy, higher-coverage feature sets than all competing methods, TGSS confirms that the topology graph reliably acquires task-specific transfer knowledge, with specialisation degree correlating directly with performance gains over single-family baselines. Feature programs transfer reliably across architecturally distinct predictors, and remain stable across LLM backbones of varying capability, confirming that gains are driven by architectural mechanisms rather than any particular model. These results collectively establish that inter-family complementarity is a learnable and exploitable quantity that existing AutoFE methods leave entirely untapped, opening promising directions for automated family schema induction, task-type-specific transfer scheduling, and extension to multi-modal tabular settings.

References

- Guozhu Dong and Huan Liu. *Feature engineering for machine learning and data analytics*. CRC press, 2018.
- Tianping Zhang, Zheyu Aqa Zhang, Zhiyuan Fan, Haoyan Luo, Fengyuan Liu, Qian Liu, Wei Cao, and Li Jian. Openfe: Automated feature generation with expert-level performance. In *International Conference on Machine Learning*, pages 41880–41901. PMLR, 2023a.
- Noah Hollmann, Samuel Müller, and Frank Hutter. Large language models for automated data science: Introducing caafe for context-aware automated feature engineering. *Advances in Neural Information Processing Systems*, 36: 44753–44775, 2023a.
- Randal S Olson and Jason H Moore. Tpot: A tree-based pipeline optimization tool for automating machine learning. In *Workshop on automatic machine learning*, pages 66–74. PMLR, 2016.
- Franziska Horn, Robert Pack, and Michael Rieger. The autofeat python library for automated feature engineering and selection. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 111–120. Springer, 2019.
- Sungwon Han, Jinsung Yoon, Sercan O Arik, and Tomas Pfister. Large language models can automatically engineer features for few-shot tabular learning. *Forty-first International Conference on Machine Learning*, 2024.
- Nikhil Abhyankar, Parshin Shojaee, and Chandan K Reddy. Llm-fe: Automated feature engineering for tabular data with llms as evolutionary optimizers. *arXiv preprint arXiv:2503.14434*, 2025.
- Nanxu Gong, Chandan K Reddy, Wangyang Ying, Haifeng Chen, and Yanjie Fu. Evolutionary large language model for automated feature transformation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 39, pages 16844–16852, 2025.
- Arthur Asuncion, David Newman, et al. Uci machine learning repository, 2007.
- Joaquin Vanschoren, Jan N Van Rijn, Bernd Bischl, and Luis Torgo. Openml: networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60, 2014.
- Noah Hollmann, Samuel Müller, Katharina Eggenberger, and Frank Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. In *NeurIPS 2022 First Table Representation Workshop*.
- Jaehyun Nam, Kyuyoung Kim, Seunghyuk Oh, Jihoon Tack, Jaehyung Kim, and Jinwoo Shin. Optimized feature generation for tabular data via llms with decision tree reasoning. *Advances in neural information processing systems*, 37:92352–92380, 2024.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- Tuan Dinh, Yuchen Zeng, Ruisu Zhang, Ziqian Lin, Michael Gira, Shashank Rajput, Jy-yong Sohn, Dimitris Papailiopoulos, and Kangwook Lee. Lift: Language-interfaced fine-tuning for non-language machine learning tasks. *Advances in Neural Information Processing Systems*, 35:11763–11784, 2022.
- Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: unbiased boosting with categorical features. *Advances in neural information processing systems*, 31, 2018.
- Multilayer perceptron (mlp). In *Geomatic approaches for modeling land change scenarios*, pages 451–455. Springer, 2017.
- Noah Hollmann, Samuel Müller, Katharina Eggenberger, and Frank Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. In *International Conference on Learning Representations 2023*, 2023b.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.
- Jeonghyun Ko, Gyeongyun Park, Donghoon Lee, and Kyunam Lee. Ferg-llm: Feature engineering by reason generation large language models. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 4211–4228, 2025.
- Xinhao Zhang, Jinghan Zhang, Banafsheh Rekabdar, Yuanchun Zhou, Pengfei Wang, and Kunpeng Liu. Dynamic and adaptive feature generation with llm. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, pages 7029–7037, 2025.

- Kun Ouyang, Haoyu Wang, and Dong Fang. Fela: A multi-agent evolutionary system for feature engineering of industrial event log data. *arXiv preprint arXiv:2510.25223*, 2025.
- Keith Burghardt, Jienan Liu, Sadman Sakib, Yuning Hao, and Bo Li. Famose: A react approach to automated feature discovery. *arXiv preprint arXiv:2602.17641*, 2026.
- João Eduardo Batista. Embedding domain-specific knowledge from llms into the feature engineering pipeline. *arXiv preprint arXiv:2503.21155*, 2025.
- Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 01 2025. doi: 10.1038/s41586-024-08328-6. URL <https://www.nature.com/articles/s41586-024-08328-6>.
- Josh Gardner, Juan C Perdomo, and Ludwig Schmidt. Large scale transfer learning for tabular data via language modeling. *Advances in Neural Information Processing Systems*, 37:45155–45205, 2024.
- Alan Arazi, Eilam Shapira, and Roi Reichart. TabSTAR: A tabular foundation model for tabular data with text fields. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=FrXHdcTEzE>.
- Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. Tabicl: A tabular foundation model for in-context learning on large data. In *International Conference on Machine Learning*, pages 50817–50847. PMLR, 2025.
- Junwei Ma, Valentin Thomas, Rasa Hosseinzadeh, Alex Labach, Hamidreza Kamkari, Jesse C. Cresswell, Keyvan Golestan, Guangwei Yu, Anthony L. Caterini, and Maksims Volkovs. TabDPT: Scaling tabular foundation models on real data. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=pIZxE0ZCId>.
- Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. Tabllm: Few-shot classification of tabular data with large language models. In *International conference on artificial intelligence and statistics*, pages 5549–5581. PMLR, 2023.
- Chao Ye, Guoshan Lu, Haobo Wang, Liyao Li, Sai Wu, Gang Chen, and Junbo Zhao. Towards cross-table masked pretraining for web data mining. In *Proceedings of the ACM Web Conference 2024*, pages 4449–4459, 2024a.
- Myung Jun Kim, Leo Grinsztajn, and Gael Varoquaux. Carte: Pretraining and transfer for tabular learning. In *International Conference on Machine Learning*, pages 23843–23866. PMLR, 2024.
- Jiahuan Yan, Bo Zheng, Hongxia Xu, Yiheng Zhu, Danny Chen, Jimeng Sun, Jian Wu, and Jintai Chen. Making pre-trained language models great on tabular prediction. In *The Twelfth International Conference on Learning Representations*.
- Tianping Zhang, Shaowen Wang, Shuicheng Yan, Li Jian, and Qian Liu. Generative table pre-training empowers models for tabular prediction. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 14836–14854, 2023b.
- Vadim Borisov, Kathrin Sessler, Tobias Leemann, Martin Pawelczyk, and Gjergji Kasneci. Language models are realistic tabular data generators. In *The Eleventh International Conference on Learning Representations*.
- Tommaso Bendinelli, Artur Dox, and Christian Holz. Exploring llm agents for cleaning tabular machine learning datasets. In *ICLR 2025 Workshop on Foundation Models in the Wild*.
- Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Evoprompt: Connecting llms with evolutionary algorithms yields powerful prompt optimizers. *arXiv e-prints*, pages arXiv–2309, 2023.
- Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Prompt-breeder: Self-referential self-improvement via prompt evolution. In *Forty-first International Conference on Machine Learning*.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106, 2026.
- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. In *First Workshop on Foundations of Reasoning in Language Models*.
- Anja Šurina, Amin Mansouri, Lars CPM Quaedy, Amal Seddas, Maryna Viazovska, Emmanuel Abbe, and Caglar Gulcehre. Algorithm discovery with llms: Evolutionary search meets reinforcement learning. In *Second Conference on Language Modeling*.

- Henrique Assumpção, Diego Ferreira, Leandro Campos, and Fabricio Murai. Codeevolve: An open source evolutionary coding agent for algorithm discovery and optimization. *arXiv preprint arXiv:2510.14150*, 2025.
- Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent, 2025. URL <https://github.com/codelion/openevolve>, 5.
- Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025.
- Parshin Shojaee, Kazem Meidani, Shashank Gupta, Amir Barati Farimani, and Chandan K Reddy. Llm-sr: Scientific equation discovery via programming with large language models. In *The Thirteenth International Conference on Learning Representations*.
- Xianliang Yang, Ling Zhang, Haolong Qian, Lei Song, and Jiang Bian. Heuragenix: Leveraging llms for solving complex combinatorial optimization challenges. *arXiv preprint arXiv:2506.15196*, 2025b.
- Xiangsen Chen, Ruilong Wu, Yanyan Lan, Ting Ma, and Yang Liu. Molevolve: Llm-guided evolutionary search for interpretable molecular optimization. *arXiv preprint arXiv:2603.24382*, 2026.
- Nikhil Abhyankar, Sanchit Kabra, Saaketh Desai, and Chandan K. Reddy. LLEMA: Evolutionary search with LLMs for multi-objective materials discovery. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=TIqzhBvCNB>.
- Fei Liu, Tong Xialiang, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *Forty-first International Conference on Machine Learning*.
- Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. Reevo: Large language models as hyper-heuristics with reflective evolution. *Advances in neural information processing systems*, 37:43571–43608, 2024b.
- Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. Evoagent: Towards automatic multi-agent generation via evolutionary algorithms. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6192–6217, 2025.
- Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. Agentsquare: Automatic llm agent search in modular design space. In *The Thirteenth International Conference on Learning Representations*.

Appendix

A Comparison with LLM-based Baseline

We compare and summarize the differences between TOPOFE and other LLM-based methods.

(i) Exploration Strategy. CAAFE explores greedily and sequentially: it proposes one candidate feature at a time, accepts or rejects it based on validation performance, and repeats for a fixed number of iterations. FeatLLM runs many independent LLM queries in parallel, but the trials never interact with one another. OCTree follows a single-trajectory iterative optimization, refining one rule at a time before moving on to the next feature. LLM-FE performs evolutionary search over a single population that is split into island buffers, with mutation and crossover realized implicitly through prompt-conditioned generation. TOPOFE evolves multiple family-specialized islands in parallel, each with explicit LLM-driven mutation and crossover operators, and adds saturation-triggered cross-island exploration on top. The essential delta is that every baseline explores either one path (CAAFE, OCTree), independent paths (FeatLLM), or interchangeable paths (LLM-FE), whereas TOPOFE is the only method whose paths are differentiated by inductive bias and actively coordinated, each island knows what kind of feature it is searching for and learns which other island to borrow from.

(ii) Search Space. CAAFE’s search space consists of unconstrained code expressions, which in practice collapse into simple arithmetic combinations. FeatLLM deliberately restricts its space to per-class threshold and membership rules over individual features. OCTree searches over natural-language rules that are subsequently converted into code, with no pre-defined operator space. LLM-FE searches the space of full feature-transformation programs, but over one undifferentiated grammar. TOPOFE also searches full transformation programs, but partitions the space into semantically typed subspaces, and additionally makes the joint compositional spaces between families reachable through hybrid synthesis. TOPOFE is the only method that decomposes the program space into typed subspaces and treats cross-family compositions as explicit, deliberately-triggered targets: LLM-FE has the same raw expressiveness but one flat grammar, FeatLLM deliberately limits itself to simple rules, and CAAFE is unconstrained in principle yet simple in practice.

(iii) Memory Management. CAAFE remembers only the accepted features and the most recent execution error, so rejected features leave no trace and near-duplicates of failed ideas can recur. FeatLLM is fully stateless, each trial is an independent LLM call with no memory across trials. OCTree retains the full optimization trajectory of prior rules, their scores, and the associated reasoning, re-sent to the LLM at every call. LLM-FE maintains a buffer of high-scoring programs, but it stores winners only. TOPOFE maintains a dual memory: an elite archive of exemplar programs, plus a Prompt Adaptation Memory that summarizes both the accepted and the rejected history of each island into a compact natural-language “prefer/avoid” signal, continuously updated during search.

(iv) Elite Management. CAAFE has no explicit archive: the currently accepted feature set effectively is the elite. FeatLLM keeps no elites at all, the final ensemble simply averages the outputs of all trials. OCTree keeps only the best-scoring rule per feature, with no population behind it. LLM-FE maintains a score-clustered elite buffer, sampled to provide in-context demonstrations, and deduplicates programs by their performance signature. TOPOFE maintains a bounded per-island elite archive that enforces structural novelty, candidates too syntactically similar to existing members are discarded, and this archive doubles as both the in-context demonstration pool and the donor material for cross-island hybridization. TOPOFE’s elites are deduplicated by structure, keeping the archive behaviorally and syntactically diverse, which matters because elites serve as crossover and hybridization parents, so redundant parents would produce redundant offspring.

(v) Optimization Objective. CAAFE feeds back a scalar performance change plus any execution errors. FeatLLM provides no iterative feedback, the LLM sees only few-shot examples. OCTree feeds back validation scores together with decision-tree reasoning expressed in natural language, giving the LLM structural knowledge about the dataset. LLM-FE feeds back validation scores alongside high-scoring programs as in-context exemplars. TOPOFE feeds back the fitness score, elite exemplars, and the distilled preference signal from its prompt memory. The key distinction within the “rich feedback” camp is that OCTree’s decision tree summarizes the data, whereas TOPOFE’s prompt memory summarizes the search experience, the two are orthogonal forms of structured feedback, and TOPOFE steers the LLM’s proposal distribution with an explicit, cumulative account of which regions of the search space have proven productive or dead.

(vi) Feedback Signal to the LLM. CAAFE maximizes validation classification performance. FeatLLM minimizes the loss of a simple model fitted on its binary rule features. OCTree minimizes validation loss for each rule independently. LLM-FE maximizes the validation score of the best program found. TOPOFE alone optimizes an explicitly multi-term objective that combines downstream performance, a redundancy penalty over pairwise feature correlations, a stability

term rewarding consistency across validation folds, and a computational cost term. Every baseline treats predictive performance as the sole criterion, whereas TOPOFE formalizes feature quality as a multi-criteria property, and this is precisely why it uniquely wins on both redundancy and subspace-coverage metrics instead of on accuracy alone.

(vii) **Parallelism and Efficiency.** CAAFE and OCTree are inherently sequential, each iteration depends on the previous one’s outcome. FeatLLM’s independent trials are trivially parallel. LLM-FE’s islands are parallelizable in principle. TOPOFE is parallel by design: island evaluations are mutually independent, so wall-clock time per generation is governed by the slowest island rather than the total number of candidates, while saturation-triggered (rather than continuous) transfer keeps coordination overhead negligible.

B Prompt Design

This section documents the prompts of TOPOFE, organized by function.

B.1 Feature-Understanding Call

```
###
<Role>
You are an scientist working on feature engineering program generation.

###
<Task>
First understand the dataset features and task.
Explicitly reason about relationships between features and the prediction target.
Assume all input features are usable; do not exclude or de-prioritize any column.
Target: {task_description}
Columns: {comma-separated columns}

###
<Output>
Return concise bullets:
1) likely feature-to-target relationships (direction, nonlinearity, thresholds, monotonic effects when plausible)
2) likely useful feature-to-feature interactions/ratios/aggregations that could improve target prediction
3) concrete transformation ideas for each major feature group (numeric, categorical, mixed)
```

B.2 Generation Context

This prompt is shared generation context that prepended to initialize, mutation, crossover, and hybrid.

```
###
<Objective>
Primary mission: "extract, transform, and select variables from raw data by generating feature engineering Python codes based on the analysis, making machine learning models more accurate and efficient."
Generation objective: create engineered features that capture both:
- feature-to-target relationships
- feature-to-feature relationships relevant to target prediction
Avoid arbitrary transformations that are not tied to target-relevant signal.
First include a concise one-sentence docstring describing the transformation intent.
```

B.3 Generation Rules

```
###
<Role>
You are a data scientist with expert knowledge about the provided dataset.
Your role is to identify and engineer informative features to solve the <Task> effectively.
```

```

###
<Instructions>
- Return Python code lines only.
- Ensure the code is executable.
- Write the BODY of a function 'generated_feature(df)'.
- Do NOT use placeholders/undefined names like member1, feature1, x, y.
- Do NOT create toy/sample dataframes; operate on provided df only.
- Avoid providing any additional descriptions outside code.
- Use valid operator calls with dataframe series arguments.
Allowed operators for this family: {family_guide}
Current family: {family_name}

```

B.4 Initialization Prompt

The initialization prompt seeds each island with a strong baseline program:

```

{prompt_context}{generation_rules}
Produce ONE initial '{family_name}' feature program.
Use at least two source columns, or an axis=1 aggregation
across many columns.
Ground it in the analysis notes; make it a strong,
self-contained baseline for this family.

```

B.5 Mutation Prompt

The mutation prompt modifies a single parent program conditioned on PAM signals:

```

{prompt_context}{generation_rules}
Family: {family_name}
Parent program:
{parent.code}

```

```

Preferred patterns (recently rewarded): {prefer}
Avoid patterns (recently rejected/redundant): {avoid}

```

Mutate the parent into a distinct, stronger '{family_name}' feature. Change its structure meaningfully swap an operator, add a stabilizing transform, or bring in another column so the output is not merely a rescaled version of the parent.

The {prefer} and {avoid} fields are populated from the prompt-memory vector $\rho_i^{(t)}$, providing explicit directional guidance derived from the island's accumulated accept/reject history (Eq. 8).

B.6 Crossover Prompt

The crossover prompt synthesizes a child from two parents within the same island:

```

{prompt_context}{generation_rules}
Family: {family_name}
Parent A:
{p1.code}

```

```

Parent B:
{p2.code}

```

Combine the most useful structural idea from each parent into one new '{family_name}' feature. Do not concatenate them verbatim; synthesize a coherent signal that improves on both and stays low-redundancy.

B.7 Cross-Island Hybrid Synthesis Prompt

The cross-island hybrid prompt is triggered by saturation detection and generates programs in the joint composition space $\mathcal{P}_i \circ \mathcal{P}_j$:

```
{prompt_context}{generation_rules}  
TARGET FAMILY: {target.family_name}  
DONOR FAMILY: {donor.family_name}
```

```
Target parent (stay in this family's style):  
{p_t.code}
```

```
Donor parent (borrow ONE idea from a different family):  
{p_d.code}
```

The target island has stagnated. Import a single useful structural idea from the donor - an operator, a normalization, or a grouping strategy, and re-express it as a '{target.family_name}' feature.
Do not copy either parent. The result must be robust, non-trivial, and structurally novel relative to the target island's existing features.